

UNIT STUDY NOTES

Numerical Computing with NumPy

NumPy is the substrate under pandas, scikit-learn and every tensor operation you will perform. This unit covers the ndarray, dtype and shape model, vectorisation, indexing, broadcasting and the performance reasoning that decides whether code scales.

 3 source notebooks 95 topic sections extracted 7 study sections

Unit Overview

Welcome to Numerical Computing with NumPy! This session is where your Python skills start turning into true data-science skills. So far you have worked with Python lists, which are flexible but slow when dealing with tens of thousands of numbers. NumPy — short for Numerical Python — is the foundational library for scientific computing. It provides the **ndarray** (n-dimensional array), a homogeneous, fixed-size grid of numbers that lives in one contiguous block of memory. Because of that layout, operations on whole arrays run in compiled C code, often tens of times faster than an equivalent Python loop.

Why does this matter? Nearly every serious Python library used for machine learning and data analysis — pandas, scikit-learn, TensorFlow, PyTorch — is built on top of NumPy's array model. When you load images, tables, or sensor data, the numbers arrive as NumPy arrays. When you train a neural network, the weights are stored and updated inside arrays. Mastering NumPy is therefore not optional for a data scientist; it is the shared language of the field.

In this session you will learn, in order:

1. What an ndarray is and how it differs from a Python list.
2. The many ways to create arrays: `np.array`, `np.arange`, `np.linspace`, `np.ones`, `np.zeros`, `np.random.random`, and `np.identity`.
3. Array attributes (`ndim`, `shape`, `size`, `dtype`, `itemsize`) that tell you everything about the structure of your data.
4. Changing data types with `astype`.
5. Elementwise arithmetic and comparison operations that need no loops.
6. Universal functions and reductions, including the crucial `axis` parameter.
7. Indexing, slicing, and iterating over arrays.
8. Reshaping, transposing, stacking, and splitting arrays.

A mental model that will serve you well: think of a NumPy array as a **grid of numbers with a fixed shape**. Operations act on the whole grid at once, and the shape — `(rows, columns)` or `(depth, rows, columns)` — defines how you step through the data. Everything else in NumPy is built from that single idea. Let's begin.

What is NumPy

Numerical Python - library for fast N-dim array ops.

Why

Vectorization, performance, memory efficiency, rich math.

How

C-backed homogeneous arrays.

AI/ML relevance

Core for data, linear algebra.

Session 13 gave you the building blocks: creating arrays, reading their shape, slicing, and reshaping. This session upgrades you from a NumPy user to a NumPy thinker. The four big ideas you will meet are speed, power indexing, broadcasting, and formulas.

First, **speed**. A Python loop touches each element through the interpreter, which is slow. A NumPy operation hands a whole array to compiled C code that walks the memory directly, so the same addition can be 50-100 times faster — and leaner in memory, because every element is stored cleanly next to its neighbours. This is why real machine learning loops are almost invisible inside NumPy calls.

Second, **power indexing**. Plain slicing picks blocks. Fancy indexing picks any scattered positions with an index array, and boolean masking picks exactly the values that satisfy a condition. These two techniques replace entire pages of filter logic with a single line and are the closest thing data scientists have to a SQL WHERE clause.

Third, **broadcasting**. When two arrays have different shapes, NumPy does not panic — it stretches the smaller one across the larger, provided the dimensions are compatible. Mastering the two broadcasting rules lets you write code for the general case and stop reshaping by hand.

Fourth, **formulas**. Machine learning is a collection of vectorised equations. The sigmoid, mean-squared-error, and binary cross-entropy functions are each a one-liner once you combine `np.exp`, `np.mean`, and elementwise arithmetic. Seeing a neural network as a chain of array formulas turns scary math into plain NumPy.

You will also handle missing data with `np.nan`, visualise formulas with matplotlib, and build coordinate grids with `meshgrid` for surface plots. Every idea here is a tool you will reuse in pandas and in every model you train. Let's begin with why NumPy wins on speed.

Sessions 13 and 14 taught you what NumPy is and how its core mechanics work. Session 15 is different in flavour: it is a tour of **25 useful functions** you will reach for again and again in real data work. These are not obscure corners of the library — they are the everyday utilities that turn ten lines of fiddly Python into one clear NumPy expression.

The format is simple. For each trick you get a one-line definition, a code cell that demonstrates it, and a short deep-dive that explains the idea, the role of `axis`, and where the function shows up in practice.

Here is a preview of what is coming:

- **Ordering and selection:** `np.sort`, `np.append`, `np.concatenate`, `np.unique`, `np.expand_dims`, `np.where`.
- **Extremes and cumulative stats:** `np.argmax` / `np.argmin`, `np.cumsum` / `np.cumprod`, `np.percentile`, `np.histogram`, `np.corrcoef`.
- **Membership and shape surgery:** `np.isin`, `np.flip`, `np.put`, `np.delete`.
- **Set-style operations:** `np.union1d`, `np.intersect1d`, `np.setdiff1d`, `np.setxor1d`, `np.in1d`.
- **Limits and extras:** `np.clip`, plus `swapaxes`, `uniform`, `count_nonzero`, `tile`, `repeat`, `allclose`.

As you work through the menu, keep two ideas in mind. The first is the **axis parameter**: almost every function in this session accepts an `axis` argument that decides whether you operate along rows (`axis=0`) or columns (`axis=1`), and getting it right is usually the difference between a correct and a surprisingly wrong result. The second is **copy-versus-view**: some functions return a brand-new array whereas others mutate the input in place, and knowing which is which prevents silent bugs.

By the end you should be able to look at a data manipulation task and immediately name the one function that solves it. That instinct — recognising the right tool on sight — is exactly what Session 15 is designed to build.

One more framing that helps: think of these twenty-five functions as a vocabulary. You already know how to write sentences in NumPy; this session expands your word bank. Just as a larger vocabulary lets you say something complicated in one clean phrase, a larger function bank lets you express any data transformation in fewer, clearer lines. Phrase-count is not the goal, but readability and speed usually follow it. Let's get started.

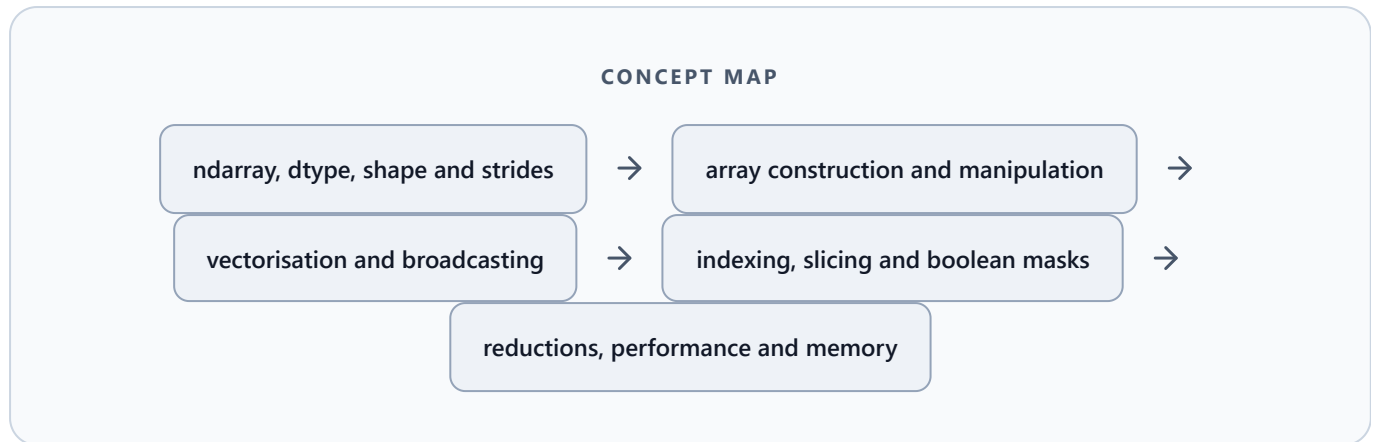
Prerequisites

- Pythonic Thinking
- Lists and indexing

What you will be able to do

- ✓ Create and reshape ndarrays and reason explicitly about shape and dtype
- ✓ Replace Python loops with vectorised array expressions
- ✓ Apply broadcasting correctly and debug silent shape bugs
- ✓ Choose reductions, indexing and memory layouts for large arrays

What This Unit Covers



The sections below follow this arc, from foundational semantics to the engineering decisions that follow from them.

Core Concepts

NumPy Fundamentals

NumPy Arrays Vs Python Sequences

- NumPy arrays have a fixed size at creation, unlike Python lists (which can grow dynamically). Changing the size of an ndarray will create a new array and delete the original.
- The elements in a NumPy array are all required to be of the same data type, and thus will be the same size in memory.
- NumPy arrays facilitate advanced mathematical and other types of operations on large numbers of data. Typically, such operations are executed more efficiently and with less code than is possible using Python's built-in sequences.
- A growing plethora of scientific and mathematical Python-based packages are using NumPy arrays; though these typically support Python-sequence input, they convert such input to NumPy arrays prior to processing, and they often output NumPy arrays.

Creating Numpy Arrays

Why NumPy

Vectorized C-backed ops, contiguous memory, broadcasting. Essential for numerical work and ML arrays.

AI/ML relevance

Shapes, dtypes, vectorization, indexing, and clean tabular ops are critical for reliable data pipelines.

np.array — basic

- `np.array(sequence)` — create a numpy array from a list/tuple

- Dimensions are decided by the depth of the nested lists
- 1D = vector, 2D = matrix, 3D = multi-dimensional tensor
- `print(a)` — very clean tabular output

PYTHON

```
import numpy as np

a = np.array([1, 2, 3])           # 1D
b = np.array([[1, 2, 3], [4, 5, 6]]) # 2D
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]]) # 3D
```

Deep Dive : NumPy vs Python lists

Numpy arrays vs Python sequences:

- Same type elements (homogeneous) — fixed dtype
- Contiguous memory block → fast + cache-friendly
- Vectorized ops (no loops) — C speed
- Broadcasting support
- Fixed size (no append like list)
- Array math element-wise (operators)
- numpy: scientific computing, ML base
- Python list: flexible, heterogeneous, mixed types
- numpy.ndarray — the workhorse data container
- Install: `!pip install numpy`

PYTHON

```
# np.array
import numpy as np

a = np.array([1,2,3])
print(a)
```

PYTHON

```
import numpy as np
a = np.array([1,2,3]) # 1D
print(a)             # [1 2 3]
```

Deep Dive : Multi-dimensional arrays

- `np.array(list)` — nested lists → N-D arrays
- 1D: vector | 2D: matrix | 3D+: tensor
- Depth = nesting level

- Shape reflects: b (2,3), c (2,2,2)
- Printing: rows = first dim, bracket nesting shows dimensions
- Data must be rectangular (no ragged)
- ragged → ValueError/object arrays
- Building complex arrays: np.arange().reshape()
- Access: b[0] row, b[0][1] / b[0,1]
- ML images: (batch, h, w, ch) — 4D

PYTHON

```
# 2D and 3D
b = np.array([[1,2,3],[4,5,6]])
print(b)
```

PYTHON

```
c = np.array([[1,2],[3,4]],[[5,6],[7,8]])
print(c)
```

dtype & arange

- `dtype=` — force the element type (int, float, ...)
- `np.arange(start, stop, step)` — numpy's version of range
- The stop value is not included (end exclusive)
- `arange().reshape()` — chaining: change the shape in one go

PYTHON

```
np.array([1, 2, 3], dtype=float) # dtype explicitly set
np.arange(1, 11, 2)             # 1,3,5,7,9
np.arange(16).reshape(2, 2, 2, 2)
```

Deep Dive : dtype & np.arange

- **dtype** — data type of elements (int32, float64, bool)
- `dtype=` param — explicit type control (memory)
- **np.arange(start, stop, step)** — like range but ndarray
- arange float values allowed
- reshape — total elements must match product of dims
- arange fast building dense arrays
- convert: `.astype(np.int32)`
- default ints → int64, floats → float64
- Common: int32/float32 for memory saving in ML

PYTHON

```
# dtype
np.array([1,2,3],dtype=float)
```

PYTHON

```
# np.arange
np.arange(1,11,2)
```

ones / zeros / random / linspace / identity

- `ones(shape)` and `zeros(shape)` accept a shape tuple and fill every cell with 1 or 0 — perfect for placeholder matrices, masks, and bias initialisation.
- `random.random(shape)` produces uniform random floats between 0 and 1; `np.random.rand` is a similar helper.
- `linspace(start, stop, n)` returns exactly `n` points evenly spaced between `start` and `stop`, both included. Use `linspace` when you know how many points you want; use `arange` when you know the step size.
- `identity(n)` builds the `n x n` identity matrix with 1s on the main diagonal.
- All of these builders accept either a single integer (1D) or a shape tuple (N-D).

PYTHON

```
np.ones((3, 4))           # a 3x4 array filled with 1s
np.zeros((3, 4))          # a 3x4 array filled with 0s
np.random.random((3, 4))  # uniform random floats in [0, 1)
np.linspace(-10, 10, 10, dtype=int) # 10 evenly spaced integers
np.identity(3)            # 3x3 identity matrix
```

Why these builders matter

A good NumPy habit is to allocate arrays with a one-line builder instead of filling a Python list with a loop.

- `ones` and `zeros` are the standard way to create blank arrays before filling them. In machine learning you will initialise weight matrices, masks, and bias vectors to zeros all the time.
- `random.random` samples uniformly from the interval `[0, 1)`. When you need a normal (Gaussian) distribution instead, reach for `np.random.randn`; a uniform sample produces a flat histogram, while a normal sample produces the familiar bell curve.
- `linspace` is count-based: `np.linspace(-10, 10, 10)` always gives exactly 10 values, `-10` through `10`. `arange` is step-based and end-exclusive: `np.arange(-10, 11, 2)` gives `-10, -8, ..., 10`. Choosing the right one depends on whether you think in terms of the number of points or the step size.
- `identity(n)` is the matrix that leaves every vector unchanged under multiplication: `I @ v == v`. It is the multiplicative identity for matrices, the same way 1 is for numbers.
- Passing `dtype=int` to `linspace` truncates the evenly spaced floats to integers, which is exactly what the code cell below demonstrates.

PYTHON

```
# np.ones and np.zeros
np.ones((3,4))
```

PYTHON

```
np.zeros((3,4))
```

Array attributes

ATTRIBUTE	MEANING
<code>ndim</code>	number of dimensions (rank)
<code>shape</code>	tuple giving the size of each axis
<code>size</code>	total elements (<code>a * b * c</code>)
<code>itemsize</code>	bytes occupied by one element
<code>dtype</code>	element data type (int32, float64, ...)

These attributes describe the structure of an array. `shape` is the most important one: it is the foundation of indexing and the first thing to check when an operation fails. If a shape does not match what you expect, stop and print `arr.shape` before anything else.

PYTHON

```
a3.ndim      # number of dimensions (rank)
a3.shape     # size of each axis, e.g. (rows, cols)
a2.size      # total number of elements
a3.itemsize  # bytes used by a single element
a3.dtype     # data type of the elements
```

Deep Dive : Array attributes

- `ndim` — rank: 1D=1, 2D=2, 3D=3
- `shape` — biggest/first check: (rows, cols), (d, h, w)
- `size` — `ndim` multiply = size
- `itemsize` — memory per element (8 for float64 when 4 for int32)
- `dtype` — int/float/bool/object/bytes
- `nbytes` — total memory = size * itemsize
- `a.shape` tuple to set/reshape
- Debugging data — always check shape first
- Inconsistent dtype bug: mixed → object (avoid)

PYTHON

```
a1 = np.arange(10, dtype=np.int32)
a2 = np.arange(12, dtype=float).reshape(3,4)
a3 = np.arange(8).reshape(2,2,2)

a3
```

PYTHON

```
# ndim
a3.ndim
```

Changing Datatype

What/Why

Variables reference objects in memory. Data types determine operations allowed and behavior.

Semantics

Python is dynamically typed. Assignment binds names to objects; mutability matters.

Common mistakes

Confusing assignment vs comparison, mutability side effects.

- `astype(type)` converts an array to a new `dtype` and returns a new array; the original remains untouched.
- Float to int truncates towards zero, so 3.9 becomes 3.
- Int to float widens the type so fractional values can be represented.
- Why bother? Lower-precision types such as `int32` or `float32` use half the memory of their 64-bit counterparts. Downcasting large datasets saves memory, and aligning the `dtype` of model inputs avoids cryptic errors later.

PYTHON

```
array.astype(np.int32)  # returns a NEW array in the new type
```

Deep Dive : Changing datatype — astype

- `astype` — `dtype` change; original untouched (immutable-ish)
- `float64` → `int32`: truncates decimals
- `int` → `float`: precision up
- memory: `float64` takes double `int32`
- usage: downcast dataset for efficiency
- object → numeric conversions reserved
- errors: 'raise'/'ignore'

- monetary precision: keep float64
- ML: model input dtype alignment
- Check result: `.dtype / .nbytes`

PYTHON

```
# astype
a3.astype(np.int32)
```

PYTHON

```
a3.astype(np.int32) # returns NEW array with converted dtype
```

Array operations — scalar & vector

- NumPy **elementwise** operations — no need to write loops
- Arithmetic (`+` `-` `*` `/` `**`), relational (`==` `>` `<`) are all vectorized
- A scalar is broadcast to every element
- An operation between two arrays needs matching shapes (broadcast rules)

PYTHON

```
a1 = np.arange(12).reshape(3,4)
a2 = np.arange(12,24).reshape(3,4)

a2
```

PYTHON

```
a2 == 15 # elementwise relational → boolean array
a1 ** 2 # scalar: har element pe
a1 ** a2 # vector: elementwise, shape match karna
```

Deep Dive : Scalar operations (vectorized)

- Scalar (single number) op → applies to EVERY element (no loop!)
- Vectorization: C-level loops, faster than Python for
- Operations: `+` `-` `*` `/` `//` `%` `**` comparisons `<` `>` `==`
- Comparison → boolean array (masks!)
- `a1 ** a2` — elementwise power with matching shapes (vector op)
- vector ops require same shape (or broadcast — covered later)
- Never write element loops unless needed
- Vectorized = default numpy style

PYTHON

```
# scalar operations

# arithmetic
a1 ** 2
```

PYTHON

```
# relational
a2 == 15
```

Array Functions

PYTHON

```
a1 = np.random.random((3,3))
a1 = np.round(a1*100)
a1
```

Array functions — axis

- Reductions: `max/min/sum/prod/mean/median/std/var`
- `axis=0` → column-wise; `axis=1` → row-wise
- Without axis → result over the whole array
- `np.sin/exp/log/ceil/floor/round` — elementwise math
- `np.dot(A, B)` — matrix multiplication (A.cols == B.rows)

PYTHON

```
np.prod(a1, axis=0) # col-wise product (axis 0)
np.var(a1, axis=1) # row-wise variance (axis 1)
np.mean(a1)        # global
```

Deep Dive : Array functions — axis

- **axis:** 0 = column-wise (vertical), 1 = row-wise (horizontal)
- exercise: `np.sum(a1, axis=0)` → per-column sums (len=cols)
- `axis=1` → per-row (len=rows)
- no axis → whole-array scalar
- functions: max/min/sum/prod/mean/median/std/var
- trig: `np.sin/cos/tan`
- dot: `a2 @ a3` matrix mult (`np.dot`)
- log/exp: `np.exp/np.log`
- rounding: `np.round/np.floor/np.ceil`
- axis selection = reduce dimension removed

PYTHON

```
# max/min/sum/prod
# 0 -> col and 1 -> row
np.prod(a1,axis=0)
```

PYTHON

```
# mean/median/std/var
np.var(a1,axis=1)
```

Indexing & slicing

- Multi-dimensional indexing uses comma-separated values: `a[r, c]` for 2D, `a[d0, d1, d2]` for 3D.
- Slicing works exactly like Python lists: `start:stop:step`, and `:` selects everything along that axis.
- Combine integer indexes and slices freely; indexing is the syntax you will use daily for random access into tables and tensors.

PYTHON

```
a1 = np.arange(10)
a2 = np.arange(12).reshape(3,4)
a3 = np.arange(8).reshape(2,2,2)

a3
```

PYTHON

```
a1
```

Deep Dive : Indexing & slicing

- Multiple indexes comma-separated per dim: `a[r, c]`
- Slicing per dim: `a[s1:e1:s1s, s2:e2:s2s, ...]`
- **Views not copies** — slices share memory (mutation propagates!)
- `:` — whole dimension
- Higher-dim slices — combine indexing+stepping
- `.reshape` view too
- Reference: change via slice → modifies original
- Use `.copy()` to detach
- Master slicing = core numpy/data skills

PYTHON

```
a2[1,0]
```

PYTHON

a3

Iterating arrays

- Iterating an ndarray with a plain `for` loop walks the **outermost axis**.
- A 2D array yields rows; a 3D array yields 2D slabs; only a 1D array yields scalars.
- `np.nditer(a)` flattens the traversal so you visit every element one by one.
- In real data pipelines, prefer vectorised functions over loops — but know how iteration works so you can inspect arrays during debugging.

PYTHON

```
for i in a1:           # 1D - yields each element
for i in a2:           # 2D - yields each row
for i in a3:           # 3D - yields each 2D block
for i in np.nditer(a3): # yields every scalar element
```

Deep Dive : Iterating arrays

- Default iteration traverses **first axis**
- 1D → items, 2D → rows, 3D → 2D blocks
- `np.nditer` — flatten to elements (`np.ravel` too)
- Vectorized preferred: `a1.sum()` not `sum(a1)`
- flatten: `.flat`, `.ravel()`, `.flatten()`
- Read: axis-wise structure
- Iteration pythonic loops slow — avoid big ones
- use apply/vectorized/functions instead
- `nditer` useful debug & custom traversal

PYTHON

a1

```
for i in a1:
    print(i)
```

PYTHON

a2

reshape / transpose / ravel

- `reshape` — change the shape while keeping the total number of elements fixed. The product of the new dimensions must match the old ones.
- `transpose` / `.T` — reverse or permute the axes (for 2D that means rows to columns).

- `ravel()` — collapse everything into one flat 1D array; it is a view when possible, so use `flatten()` if you want a guaranteed copy.
- After reshaping, all indexing and slicing is done on the new shape.

PYTHON

```
np.transpose(a2)  # swap rows and columns
a2.T              # shorthand for the same
a3.ravel()        # flatten into a 1D view
```

Deep Dive : reshape / transpose / ravel

- transpose — reverse axes (matrix transpose)
- `a.T` is the fastest notation
- reshape — same data new shape (view if contiguous)
- ravel — flatten to 1D (view where possible)
- `flatten()` — always copy
- views: change affects source
- reshape requires size match
- -1 auto: `a.reshape(-1)`, `a.reshape(3, -1)`
- ML: feature matriciation, model input shape
- Euler: `.swapaxes`, `.moveaxis`

PYTHON

```
# Transpose
np.transpose(a2)
a2.T
```

PYTHON

```
# ravel
a3.ravel()
```

Stacking — concatenate

- `hstack` joins arrays along the column direction — the number of rows must match, and the result has more columns.
- `vstack` joins arrays along the row direction — the number of columns must match, and the result has more rows.
- Shape consistency matters: mismatched shapes raise a `ValueError`.
- Stacking is how you combine new features (`hstack`) or new observations (`vstack`) in a data pipeline.

PYTHON

```
np.hstack((a4, a5))  # side by side: columns are joined
np.vstack((a4, a5))  # one below the other: rows are joined
```

Deep Dive : Stacking — concatenate

- `hstack` — horizontal: same rows, columns add (along axis 1)
- `vstack` — vertical: rows add (axis 0)
- `concatenate(axis=)` — general combine
- dimensions must align on non-joining axis
- stacking appends dataset columns/rows (feature join)
- arrays for joining — datasets more observations/features
- setup: `a4, a5` shape (3,4) both
- Misalignment → `ValueError`
- For lists of arrays: `np.stack([a,b,c])` new axis

PYTHON

```
# horizontal stacking
a4 = np.arange(12).reshape(3,4)
a5 = np.arange(12,24).reshape(3,4)
a5
```

PYTHON

```
np.hstack((a4,a5))
```

Splitting

- `hsplit(arr, n)` — divide along the columns
- `vsplit(arr, n)` — divide along the rows
- Result is a list of arrays — index with `[0]`, `[1]`
- Useful for train/test splits and batch processing

PYTHON

```
np.hsplit(a4, 5)  # 5 vertical slices (column groups)
np.vsplit(a5, 2)  # 2 horizontal slices (row groups)
```

Deep Dive : Splitting

- `hsplit` — divide along the columns (split features)
- `vsplit` — divide along the rows (samples)
- Even split required — a remainder → error
- Returns list of sub-arrays

- `split(axis=)` general
- Real: train/test split, feature sets
- chunks for batch processing
- arrays preserve dtype/shape
- asymmetric: slice indexing for irregular
- arithmetic guard: total divisible

PYTHON

```
# horizontal splitting
a4
```

PYTHON

```
np.hsplit(a4,5)
```

Summary

In Session 13 you covered the core building blocks of NumPy. You created arrays from lists with `np.array`, generated numeric sequences with `np.arange` and `np.linspace`, and used builders such as `np.ones`, `np.zeros`, `np.random.random`, and `np.identity`. You inspected arrays through `ndim`, `shape`, `size`, `itemsize`, and `dtype`, switched types with `astype`, and saw that arithmetic runs elementwise across whole arrays without explicit loops. Reductions with an `axis` argument collapse rows or columns at C speed, while indexing, slicing, and iteration give you precise access to any subset of the data. Finally you reshaped, transposed, flattened, stacked, and split arrays. These fundamentals — vectorisation, shapes, and indexing — are exactly what you will use every day in data science and machine learning.

Session 14 moved you past the basics into the ideas that make NumPy genuinely powerful. You saw why vectorised operations beat Python loops by roughly 50x and why a contiguous `dtype`-controlled array uses far less memory than a list. On the indexing side you mastered fancy indexing with index arrays and boolean masking for filtering. The broadcasting rules let the smaller array stretch automatically during arithmetic, removing the need for explicit replication in everything from the same-shape case to `(1,3) + (4,1)`. You translated real machine-learning formulas — sigmoid and MSE — into vectorised one-liners, handled missing values with `np.nan`, visualised functions with matplotlib, and used `meshgrid` to build coordinate grids. Combined with Session 13, you now have the NumPy vocabulary used across data science and deep learning.

Session 15 stitched together a practical toolkit of twenty-five NumPy functions. You sorted with `np.sort`, grew arrays with `np.append` and `np.concatenate`, and deduplicated with `np.unique`. `np.expand_dims` adjusted shapes for broadcasting while `np.where` performed vectorised conditionals in one call. Extremes and running aggregates came from `np.argmax` / `np.argmin` and `np.cumsum` / `np.cumprod`, and distribution insight from `np.percentile`, `np.histogram`, and `np.corrcoef`. Membership filtering used `np.isin`; reversing, targeted replacement, and removal used `np.flip`, `np.put`, and `np.delete`. The set functions gave you union, intersection, difference, and membership in one family, and `np.clip` bounded values between limits. Closing utilities — `swapaxes`, `uniform`, `count_nonzero`, `tile`, `repeat`, `allclose` — completed the set. Keep this menu at hand: half of fluent NumPy is simply knowing which function already exists.

Advanced NumPy

Advanced NumPy — Speed, Indexing, Broadcast

Topic: 06.02 · **Level:** Intermediate · **Type:** THEORY + CODE

Numpy vs Python lists — speed

- NumPy is **C/CPU optimized** — ~50-100x faster than a pure loop
- This code is measured with `time.time()`
- A vectorized operation runs in a single pass, no loop involved
- In data science, this is exactly what makes the difference on large arrays

PYTHON

```
import numpy as np
import time

# Python List addition (Loop)
c = [x + y for x, y in zip(a, b)]

# Numpy vectorized
a = np.arange(10000000)
c = a + b
```

Deep Dive : Speed — vectorization wins

- Vectorized ops run at C speed (SIMD + no Python loop)
- Same operation: numpy ~50-100x faster than Python list loop
- Reason: tight C loop + contiguous memory
- Python per-element creates interpreter overhead
- Approx 3.26s vs 0.06s (~54x)
- CPU: numpy uses optimized BLAS/LAPACK in many ops
- Rule: never element-loop when vector op exists
- This scale: 10M element addition instant
- ML data millions row — numpy is the backbone

PYTHON

```
# speed
# List
a = [i for i in range(10000000)]
b = [i for i in range(10000000, 20000000)]

c = []
import time

start = time.time()
for i in range(len(a)):
    c.append(a[i] + b[i])
print(time.time()-start)
```

PYTHON

```
# numpy
import numpy as np
a = np.arange(10000000)
b = np.arange(10000000, 20000000)

start = time.time()
c = a + b
print(time.time()-start)
```

Memory usage

- List — each element is its own object (lots of overhead)
- NumPy array — contiguous memory, a single type
- `dtype=np.int8` → 1 byte per element vs 28+ bytes in a list
- Compare with `sys.getsizeof()` — numpy makes a big difference

PYTHON

```
import sys
sys.getsizeof([...])          # List: huge
sys.getsizeof(np.arange(..., dtype=np.int8)) # numpy: compact
```

Deep Dive : Memory efficiency

- List: pointer (8) + int object (~28) + list overhead — huge
- Numpy: raw C array tight packed — minimum
- dtype choice = memory control (int8 min, float64 8 bytes)
- int8 → 1 byte; float32 4; float64 8
- Contiguous = cache-friendly fast
- numpy memory ~= size * itemsize only
- Efficiency: store 10M ints → MB not GB
- Convenience: homogeneous type assumption

- ML models train on (N, features) numpy — compact

PYTHON

```
# memory
a = [i for i in range(10000000)]
import sys

sys.getsizeof(a)
```

PYTHON

```
a = np.arange(10000000, dtype=np.int8)
sys.getsizeof(a)
```

Advanced Indexing

PYTHON

```
# Normal Indexing and slicing

a = np.arange(24).reshape(6,4)
a
```

PYTHON

```
a[1,2]
```

Fancy indexing

- Normal slicing selects a *range* of positions (`1:3`).
- **Fancy indexing** selects *arbitrary* positions using a list or array of indices such as `[0, 2, 3]`.
- The index list can be in any order, does not need to be sorted, and may repeat values.
- Unlike a slice, the result of fancy indexing is a **copy**, not a view — later changes to it will not affect the original array.

PYTHON

```
a = np.arange(24).reshape(6, 4)
a[:, [0, 2, 3]] # select columns 0, 2 and 3 with a List
```

Deep Dive : Fancy indexing — index arrays

- Fancy indexing: index/pick arbitrary positions with INDEX ARRAYS
- Result is a **COPY** (unlike slicing views)
- List/array of indices → gather elements
- Row/col: `a[rows, cols]` tag cross
- Boolean masks alongside (next)
- Use: reorder rows, subsample, feature select

- `a[[0,1],[1,2]]` → `(a[0,1], a[1,2])` pair-wise
- Different from slices — fancy = copy semantics
- DataFrame `.iloc` same concept later
- Powerful data extraction tool

PYTHON

Fancy Indexing

`a[:,[0,2,3]]`

PYTHON

```

a = np.arange(24).reshape(6,4)
a[:,[0,2,3]]      # select specific COLUMNS (0,2,3) - copy
a[[1,2,0],:]      # specific rows

```

Boolean / mask indexing

- A comparison produces a boolean array of the same shape — the **mask**.
- Placing the mask inside brackets, `a[mask]`, returns only the elements where the mask is True.
- Combine conditions with `&` (and), `|` (or), and `~` (not) — never Python's `and` / `or`, which do not broadcast over arrays.
- Surround every condition in parentheses when combining them.
- This is NumPy's equivalent of a SQL WHERE clause and your go-to tool for filtering data.

PYTHON

```

a = np.random.randint(1, 100, 24).reshape(6, 4)

a[a > 50]          # keep every element greater than 50
a[a % 2 == 0]      # keep even numbers
a[(a > 50) & (a % 2 == 0)] # combine several conditions
a[~(a % 7 == 0)]   # keep elements not divisible by 7

```

Deep Dive : Boolean / mask indexing

- Boolean mask = same-shape bool array
- `a[mask]` → elements where mask True (1D flat result)
- Conditions produce mask: `a>50`, `a%2==0`
- Combine: `&` (and), `|` (or), `~` (not) — NOT python and/or
- Parentheses required around conditions
- Masking = most common filtering tool
- Write: `a[(a>50) & (a<90)]` ranged
- Sets membership by mask (`np.isin` later)
- Result always 1D flattened selection

- Critical: & not and — numpy error otherwise

PYTHON

```
# Boolean Indexing
a = np.random.randint(1,100,24).reshape(6,4)
a
```

PYTHON

```
# find all numbers greater than 50
a[a > 50]
```

Broadcasting

The term broadcasting describes how NumPy treats arrays with different shapes during arithmetic operations.

The smaller array is “broadcast” across the larger array so that they have compatible shapes.

Broadcasting — same shape

- Same shape → direct elementwise, no magic involved
- Broadcasting comes into play when the shapes DIFFER

PYTHON

```
a = np.arange(6).reshape(2, 3) # (2,3)
b = np.arange(6, 12).reshape(2, 3) # (2,3)
a + b # elementwise direct
```

Deep Dive : Broadcasting — same shape

- Broadcasting = elementwise ops between compatible shapes
- Same shape → direct elementwise, no copy
- Compares shape dim-by-dim
- Rule 1: same dims → fine
- $a + b \rightarrow c[i,j] = a[i,j] + b[i,j]$
- Vectorized — fast
- Diff shape needs broadcast rule (covered later)
- Matrices: shapes equal or 1 / missing
- Alignment: trailing dims first
- Works for + - * / ** etc.
- numba-vectorized intuitive

PYTHON

```
# same shape
a = np.arange(6).reshape(2,3)
b = np.arange(6,12).reshape(2,3)

print(a)
print(b)

print(a+b)
```

PYTHON

```
a = np.arange(6).reshape(2,3)
b = np.arange(6,12).reshape(2,3)
a + b      # elementwise - shapes match directly
```

Broadcasting — different shapes

- When shapes differ, NumPy **stretches** the smaller array so both end up compatible.
- Here `b` has shape `(1, 3)`; its single row is replicated until `b` behaves like `(2, 3)` before the addition.
- No loops are written — the operation stays fully vectorised.
- Stretching only works when every dimension of the two arrays matches or is 1; the formal rules follow below.

PYTHON

```
a = np.arange(6).reshape(2, 3)      # (2, 3)
b = np.arange(3).reshape(1, 3)      # (1, 3)
a + b      # b is stretched to (2, 3) and added elementwise
```

Deep Dive : Broadcasting — different shapes

- $a(2,3) + b(3,) \rightarrow b$ broadcast to $(2,3)$
- Trailing dimensions align; dims of size 1 stretch
- Rule 2: dims either equal or one of them is 1
- Rules summary:
 1. align trailing dims
 2. minus-minus: equal | 1 stretches | missing treated 1
 3. result shape = max dims
- Examples:
 - $(3,4) + (4,) \checkmark$
 - $(3,4) + (1,4) \checkmark$
 - $(3,1)+(1,4) \rightarrow (3,4) \checkmark$
 - $(3,4)+(4,3) \times$ mismatch
 - $(4,4)+(2,2) \times$

- Scalar broadcast: $(2,3)+5 \rightarrow$ all
- Concept: no explicit loops — implicit replication
- Widely used w/ mean subtract, scaling, neural nets bias

PYTHON

```
# diff shape
a = np.arange(6).reshape(2,3)
b = np.arange(3).reshape(1,3)

print(a)
print(b)

print(a+b)
```

PYTHON

```
a = np.arange(6).reshape(2,3) # (2,3)
b = np.arange(3).reshape(3)   # (3,)
a + b      # b stretches along rows → each row + b
```

Broadcasting Rules

1. Make the two arrays have the same number of dimensions.

- If the numbers of dimensions of the two arrays are different, add new dimensions with size 1 to the head of the array with the smaller dimension.

2. Make each dimension of the two arrays the same size.

- If the sizes of each dimension of the two arrays do not match, dimensions with size 1 are stretched to the size of the other array.
- If there is a dimension whose size is not 1 in either of the two arrays, it cannot be broadcasted, and an error is raised.

SOURCE DIAGRAM

[open the original figure](#)

Broadcasting rules quick reference

2 rules:

1. Right-align the shapes; every dim must be **1** or equal
2. Dims of size **1** get stretched

SHAPE	COMPATIBLE?
$(4,3) + (3,)$	✓
$(3,4) + (3,)$	✗
$(1,3) + (3,1)$	✓ ($\rightarrow (3,3)$)
$(4,4) + (2,2)$	✗

PYTHON

More examples

```
a = np.arange(12).reshape(4,3)
b = np.arange(3)

print(a)
print(b)

print(a+b)
```

PYTHON

```
a = np.arange(12).reshape(3,4)
b = np.arange(3)

print(a)
print(b)

print(a+b)
```

Working with mathematical formulas

Loops

for iterates over iterables; while runs while condition true. Use break/continue/else appropriately.

When to use

Repetitive tasks, processing collections.

PYTHON

```
a = np.arange(10)
np.sin(a)
```

Math formulas — vectorized

- In ML, every formula is written in a vectorized way on arrays
- sigmoid, MSE, BCE — a combination of `np.exp`, `np.mean` and elementwise ops
- Direct array math instead of loops — 1000x cleaner and faster

- Binary cross-entropy is also similar to `np.mean((a - p)**2)`

PYTHON

```
def sigmoid(array):
    return 1 / (1 + np.exp(-(array)))

a = np.arange(100)
sigmoid(a)

def mse(actual, predicted):
    return np.mean((actual - predicted) ** 2)
```

Deep Dive : Math formulas — vectorized

- ML formulas = numpy one-liners (no loops)
- sigmoid: e^{-x} vectorized over array
- MSE: diff → square → mean — all array ops
- Binary cross entropy similar
- np.array ops handle any shape
- Implicit broadcasting int enrichment
- These ARE model math functions — real usage
- Vectorization = numpy codebase style
- Larger scale: same functions on matrices
- Prefer np ops over manual loops in ML code

PYTHON

```
# sigmoid
def sigmoid(array):
    return 1/(1 + np.exp(-(array)))

a = np.arange(100)

sigmoid(a)
```

PYTHON

```
# mean squared error

actual = np.random.randint(1,50,25)
predicted = np.random.randint(1,50,25)
```

Missing values — np.nan

- `np.nan` (Not a Number) is NumPy's marker for missing data in float arrays.
- `np.isnan(a)` builds a mask that is True exactly where values are missing.

- Negating with `~` flips the mask, and `a[mask]` then yields a clean array without the missing entries.
- This is the standard data-cleaning step you will run before feeding an array into a model.

PYTHON

```
a = np.array([1, 2, 3, 4, np.nan, 6])
a[~np.isnan(a)]    # keep only the valid (non-NaN) values
```

Deep Dive : Missing values — np.nan

- **np.nan** — Not a Number (missing marker)
- nan propagates: any op with nan → nan (careful)
- Detect: `np.isnan(a)` → bool mask
- Remove/filter: `a[~np.isnan(a)]`
- Functions with nan: `np.nanmean`, `np.nansum`, etc.
- nan replaces None in numeric arrays (float only)
- int arrays can't hold nan (dtype object needed)
- Fill: `np.nanonum(x, nan=0)`
- DataFrame uses NaN too (pandas) — cornerstone
- Check any: `np.isnan(a).any()`

PYTHON

```
# Working with missing values -> np.nan
a = np.array([1, 2, 3, 4, np.nan, 6])
a
```

PYTHON

```
a[~np.isnan(a)]
```

Plotting with numpy + matplotlib

- `np.linspace` — smooth x points
- Any formula → vectorized → `plt.plot(x, y)`
- `y = x`, `y = x**2`, `y = np.sin(x)`, `y = x*np.log(x)`, sigmoid
- Numpy + matplotlib = visual math

PYTHON

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-10, 10, 100)    # 100 points
y = x ** 2                       # vectorized formula
plt.plot(x, y)
```

Deep Dive : Plotting with numpy + matplotlib

- matplotlib consumes numpy arrays directly
- linspace — smooth curves (100+ points)
- Visualize function shapes: parabola, sine, logistic
- plt.plot(x,y) — line chart
- plt.scatter/figure customization later (Section 08)
- Model functions vis: sigmoid S-shape
- numpy→plot = standard analytic workflow
- Multiple: plt.plot(x, y1); plt.plot(x, y2)
- labels/title optional

PYTHON

```
# plotting a 2D plot
# x = y
import matplotlib.pyplot as plt

x = np.linspace(-10,10,100)
y = x

plt.plot(x,y)
```

PYTHON

```
# y = x^2
x = np.linspace(-10,10,100)
y = x**2

plt.plot(x,y)
```

Meshgrid — the formula for building 2D grids

- `meshgrid` — builds a coordinate grid from 2 vectors
- X has repeating rows, Y has repeating columns
- The base for 3D surface plots / contour plots
- `plt.contour`, `ax.plot_surface` work on this grid

PYTHON

```
X, Y = np.meshgrid(x, y)
```

Deep Dive : Meshgrid — 2D grid

- meshgrid — 1D x, y → 2D coordinate grids
- $X[i,j]$ = x values repeated rows
- $Y[i,j]$ = y values repeated columns

- Mesh = every (xi, yi) pair — grid of coordinates
- Use: 2D functions `z = f(x,y)` surfaces, contour, contourf
- Also: vectorized eval on all grid points
- Interpolation, heatmap plotting
- pair with plt.contourf/plot_surface
- Evaluate formula at every grid cell without X-loop
- Numpy grid = scientific computation essential

PYTHON

```
X, Y = np.meshgrid(np.arange(0,10), np.arange(0,10))  
# X: x repeated rows / Y: y repeated cols  
# grid coordinates for 2D surface plots
```

NumPy Tricks

NumPy Tricks — 25 Useful Functions

Topic: 06.03 · **Level:** Intermediate · **Type:** THEORY + CODE

Session format

For every trick: definition → example in a `# code` cell

Quick menu:

FUNCTION	PURPOSE
<code>np.sort</code>	sorted copy
<code>np.append</code>	append at the end
<code>np.concatenate</code>	join arrays
<code>np.unique</code>	remove duplicates
<code>np.where</code>	condition-based select/replace
<code>np.argmax/argmin</code>	index of max/min
<code>np.cumsum/cumprod</code>	running sum/product
<code>np.percentile</code>	percentile
<code>np.histogram</code>	frequency bins
<code>np.corrcoef</code>	correlation
<code>np.isin</code>	membership filter
<code>np.flip</code>	reverse
<code>np.put</code>	element replace
<code>np.delete</code>	remove element
set funcs	union/intersect/diff
<code>np.clip</code>	limit values

The rest (swapaxes, uniform, tile, repeat, allclose) are covered at the end. Always remember the role of `axis` in each one — `0` for columns, `1` for rows.

PYTHON

```
# code
import numpy as np
a = np.random.randint(1,100,15)
a
```

PYTHON

```
b = np.random.randint(1,100,24).reshape(6,4)
b
```

Deep Dive : `np.sort` — sorting arrays

- `np.sort` — **returns sorted copy** (original untouched)
- vs `a.sort()` — in place, returns None
- `[::-1]` on result — descending

- axis — sort direction (0 cols, 1 rows)
- 1D default ascending
- string/object sort possible
- stable sort option kind arg
- unique + sort linkage
- np.argsort — indices of sorted (for ranking)
- Real: ranking, data prep

PYTHON

```
np.sort(a)[::-1]
```

PYTHON

```
np.sort(b,axis=0)
```

np.append

`np.append` adds values to the end of an array. It accepts the array, the values to add, and an optional `axis`. Without an axis the array is flattened first and the new values are simply tacked on, so the result has one more element. With `axis=1` the function appends a whole column provided the row counts match, and with `axis=0` it appends rows. Important: `np.append` always **returns a new array** — it never modifies the original — and because it copies data it is comparatively slow in loops, so prefer building a list and converting once when appending many times.

Deep Dive : np.append

- Append — **returns new array** (costly; copy)
- axis=None → flatten + append
- single element append
- axis=0 rows append (must shape match)
- axis=1 add column
- Original unchanged
- For repeated appends prefer lists then `np.array`
- dims alignment required
- int/float coercion to common dtype
- panel note: append can be slow bulk — plan

PYTHON

```
# code
np.append(a,200)
```

PYTHON

b

np.concatenate

`np.concatenate` joins a sequence of arrays along an existing axis. It takes a tuple of arrays and an `axis` parameter, and every array must match in every dimension except the one being joined. `axis=0` stacks along rows, `axis=1` along columns; `hstack` and `vstack` are convenient wrappers for the same operation. The result is always a new combined array. This is the workhorse for assembling matrices out of pieces and the NumPy counterpart of pandas `concat`.

PYTHON

```
# code
c = np.arange(6).reshape(2,3)
d = np.arange(6,12).reshape(2,3)

print(c)
print(d)
```

Deep Dive : np.concatenate

- `concatenate((a,b,...), axis)` — join along axis
- `axis=0` → rows (samples), `axis=1` → cols (features)
- Must match along other axes
- List/tuple of arrays
- vs `hstack/vstack` convenient wrappers
- Efficient bulk row combine
- Dataframes merge same concept (pandas)
- returns new array
- Use for matrix assembly

PYTHON

```
np.concatenate((c,d),axis=0)
```

PYTHON

```
np.concatenate((c,d),axis=1)
```

np.unique

`np.unique` returns the distinct values of an array, sorted ascending. Used alone it cleans duplicates; with `return_counts=True` it also reports how many times each unique value appears, which gives you a frequency table. With `return_index=True` it returns the first positions of the unique values, and with the `axis` argument it can locate unique rows or columns of a 2D array instead of unique scalars. Deduplication and value-counting are two of the most common data-cleaning jobs, and `np.unique` does both in one call.

PYTHON

```
# code
e = np.array([1,1,2,2,3,3,4,4,5,5,6,6])
```

Deep Dive : np.unique — distinct values

- Returns sorted unique elements (copy)
- `return_counts=True` → frequency
- `return_index` → first occurrence positions
- 2D: axis param unique rows
- Categories / one-hot prep
- vs set analog: `set(e)`
- freq distribution fast
- `np.unique(a, return_counts=True)` → values, counts
- for pandas: `value_counts` later
- data cleaning for duplicates

PYTHON

```
np.unique(e)
```

PYTHON

```
e = np.array([1,1,2,2,3,3,4,4,5,5,6,6])
np.unique(e)      # [1 2 3 4 5 6] — sorted unique
```

np.expand_dims

`np.expand_dims` inserts a new axis of size 1 at a chosen position. Shape `(15,)` becomes `(1, 15)` with `axis=0` or `(15, 1)` with `axis=1` — same data, new shape. That extra dimension is what makes broadcasting work: a row vector needs a leading 1, a column vector needs a trailing 1. You will reach for this whenever a function expects `(1, n)` or `(n, 1)` input, such as feeding one sample into a model or subtracting a column mean. `a[:, None]` and `a[None, :]` are equivalent shorthand.

PYTHON

```
# code
a.shape
```

Deep Dive : np.expand_dims — add axis

- Add singleton dimension at given axis
- `axis=0` → `(1,N)`; `axis=1` → `(N,1)`
- Needed: broadcasting, ML batch dims
- Broadcasting: `(N,1) + (M,)` interactions

- transpose pairing
- `np.newaxis` alias: `a[None, :]`, `a[:, None]`
- ML: model input (1, features) shape
- `reshape(-1,1)` does same column trick
- Cheap view — no copy
- Important data shape manipulation

PYTHON

```
np.expand_dims(a,axis=0).shape
```

PYTHON

```
np.expand_dims(a,axis=1)
```

np.where

`np.where` is NumPy's vectorised conditional. Called with one argument — just a condition — it returns the indices where the condition is True. Called with three arguments — a condition, a value/array for True, and a value/array for False — it returns a new array built by choosing between the two at every position. This turns an elementwise if-else written by hand into a single vectorised call, which is dramatically faster on large arrays. It is the closest NumPy thing to a functional 'replace where'.

PYTHON

```
a
```

Deep Dive : np.where — conditional select

- Two modes:
 1. `np.where(cond)` → indices (tuple of arrays)
 2. `np.where(cond, x, y)` → array: True→x, False→y
- Index mode: `a[np.where(a>50)[0]]` ≈ mask
- Fancy: positional finding
- Vectorized if/else — no loop!
- replace values based on condition
- Nested: `np.select` / boolean combo
- pandas `.where` similar
- Critical utility for conditional ops

PYTHON

```
# find all indices with value greater than 50
np.where(a>50)
```

PYTHON

```
# replace all values > 50 with 0
np.where(a>50,0,a)
```

np.argmax and np.argmin

`np.argmax` returns the index of the maximum value, and `np.argmin` the index of the minimum — not the value itself. Flat (no axis) returns a single position; with `axis=0` it reports, for each column, which row holds that column's maximum; with `axis=1` it reports, for each row, which column holds that row's maximum. Retrieving the actual value is just `a[np.argmax(a)]` (or `a.max()`). Deep learning leans on this constantly: the predicted class is `np.argmax` over the probability vector. On ties, the first occurrence wins.

PYTHON

```
# code
a
```

Deep Dive : np.argmax / np.argmin

- Returns INDEX of max/min value (not value)
- axis variant along dimension
- ML: `argmax(softmax probs)` → predicted class
- tie → first occurrence
- Get value: `a[np.argmax(a)]`
- `axis=0` → for each col which row max
- `axis=1` → for each row which col max
- model predictions = `argmax` logic
- essential deep learning metric

PYTHON

```
np.argmax(a)
```

PYTHON

```
b
```

np.cumsum and np.cumprod

`np.cumsum` computes a running total: element `i` of the output is the sum of the input through position `i`. `np.cumprod` does the same with products. Without an axis, the array is flattened and one cumulative sequence is produced; with an axis the accumulation runs along that dimension, so `axis=1` gives cumulative sums along each row. Cumulative sums are everywhere — cumulative revenue, running scores, and survival-style analyses — and cumulative products model successive multiplication, such as probability chains.

PYTHON

```
a
```

Deep Dive : `np.cumsum` / `np.cumprod`

- `cumsum` — cumulative (running) sum
- `cumprod` — running product
- axis variants along dim
- Real: daily balances, running scores
- `array[i] = sum(a[:i+1])`
- `cumprod` use probabilities chains
- compare range-less vs normal functions
- 2D axis controll
- numpy daily time series

PYTHON

```
np.cumsum(a)
```

PYTHON

```
b
```

`np.percentile`

`np.percentile` finds the value below which a given percentage of the data falls. The 50th percentile is the median, the 25th and 75th are the quartiles, and the gap between them is the interquartile range. You can request several at once by passing a list like `[25, 50, 75]`. Percentiles are robust summaries because a single weird outlier barely moves them, unlike the mean — which is why box plots and outlier sieves are built from them. `np.median(a)` is a handy alias for the 50th percentile.

PYTHON

```
a
```

Deep Dive : `np.percentile` — quantiles

- `percentile(data, q)` — value below q% of data
- `q=50` → median
- `q=25/75` → quartiles (IQR)
- Multi q: `np.percentile(a, [25,50,75])`
- `np.median` alias for `q=50`
- `np.quantile` scaled 0..1
- Robust stats (outliers resistance)

- Box plots derive from percentiles
- Very common EDA metric

PYTHON

```
np.percentile(a,50)
```

PYTHON

```
np.median(a)
```

np.histogram

`np.histogram` counts how many values fall into each bin. Pass `bins` either as a count (NumPy chooses the bin edges) or as a list of explicit edges — `[0, 50, 100]` creates two bins of width 50. The function returns a tuple `(counts, edges)`: the first element holds the frequency in each bin, the second holds the boundary values. A histogram is the numerical skeleton under every data-vs-frequency visualisation and is the fastest way to understand the shape of your data.

PYTHON

```
# code
a
```

Deep Dive : np.histogram

- Frequency count into bins
- bins edges define ranges
- Returns (*histvalues*, *binedges*)
- bins: int count or list of edges
- density=True → probability
- Hist plots built on this (plt.hist)
- Data distribution visualization data
- bin pick matters (interpretation)
- vs np.bincount (int bins only)
- EDA: understand distribution

PYTHON

```
np.histogram(a,bins=[0,50,100])
```

PYTHON

```
np.histogram(a, bins=[0,50,100])
# → counts in each bin + bin edges
```

np.corrcoef

`np.corrcoef` computes the Pearson correlation matrix for one or more arrays supplied as rows. For two series `salary` and `experience` it returns a 2x2 matrix: the diagonal is 1.0 (every variable correlates perfectly with itself) and the off-diagonal is their correlation coefficient r . A coefficient near +1 means 'move together', near -1 means 'move opposite', and near 0 means 'no linear relationship'. Because correlations feed feature selection, multicollinearity checks, and basic EDA, this little function is a staple of any modelling workflow.

Deep Dive : np.corrcoef — correlation

- Pearson correlation coefficient matrix
- $r \in [-1,1]$: strength + direction
- $r \approx 1$ positive strong, ≈ -1 negative, ≈ 0 none
- Interpretation: salary increases w/ experience?
- Matrix: `[[var1var1, var1var2], [var2var1, var2var2]]`
- $r = \text{cov}(x,y)/(\text{std}x*\text{std}y)$
- Data insight: feature relationships
- Pre ML: correlation screening
- Spearman: scipy alternative
- `np.dot` normalization manual possible

PYTHON

```
salary = np.array([20000, 40000, 25000, 35000, 60000])
experience = np.array([1, 3, 2, 4, 2])

np.corrcoef(salary, experience)
```

PYTHON

```
salary = np.array([20000, 40000, 25000, 35000, 60000])
experience = np.array([...])
np.corrcoef(salary, experience)
# → correlation matrix (1 diag, off-diag r)
```

np.isin

`np.isin(a, b)` checks, for every element of `a`, whether it also appears anywhere in `b`, and returns a boolean mask of the same shape as `a`. Combining that mask with fancy indexing — `a[np.isin(a, items)]` — filters `a` down to exactly the members of `items`. This is the vectorised answer to repeated `in` checks and is how you subset a dataset by an ID list, keep only category values you care about, or find rows whose key belongs to a reference table.

PYTHON

```
# code
a
```

Deep Dive : np.isin — membership

- `np.isin(a, b)` — elementwise "is in" → bool mask
- filters array to those values present in b
- vectorized membership (like in but array)
- mask combine: `a[mask]`
- vs `np.in1d` (1D only) — `isin` general
- data subset by id list — common
- categorical filter bricks
- preserves order in a
- duplicates show multiple
- clean kwarg `min_membership`

PYTHON

```
items = [10,20,30,40,50,60,70,80,90,100]

a[np.isin(a,items)]
```

PYTHON

```
items = [10,20,30,40,50,60,70,80,90,100]
a[np.isin(a, items)]      # elements of a that are in items
```

np.flip

The `numpy.flip()` function reverses the order of array elements along the specified axis, preserving the shape of the array.

<https://numpy.org/doc/stable/reference/generated/numpy.flip.html>

PYTHON

```
# code
a
```

Deep Dive : np.flip — reverse

- Reverse order along axis
- `axis=0` rows, `axis=1` cols, default all
- View (no copy) for 1D/trailing
- `a[::-1]` equivalent
- 2D flip = mirror
- Data ordering reverse (recent→older)
- Images: flip transformations
- keep dims; change sequence

- flipud/flipr — up-down / left-right
- Useful augmentation data

PYTHON

```
np.flip(a)
```

PYTHON

```
b
```

np.put

The `numpy.put()` function replaces specific elements of an array with given values of `p_array`. Array indexed works on flattened array.

<https://numpy.org/doc/stable/reference/generated/numpy.put.html>

PYTHON

```
# code
a
```

Deep Dive : np.put — replace values

- `put(indices, values)` — replace at given flat indexes
- works on flattened view (flat index over all dims)
- indices list + values list
- `a[0]=110` (flat pos 0), `a[1]=530`
- in place; repeats values if fewer
- mode 'clip'/'wrap' behavior
- vs assignment `a[[0,1]] = [..]`
- Useful filling specific positions
- limited by flat order (not axis)
- fast low-level assignment

PYTHON

```
np.put(a, [0,1], [110,530])
```

PYTHON

```
np.put(a, [0,1], [110,530]) # set a[0]=110, a[1]=530
```

np.delete

The `numpy.delete()` function returns a new array with the deletion of sub-arrays along with the mentioned axis.

<https://numpy.org/doc/stable/reference/generated/numpy.delete.html>

AI/ML relevance

Shapes, dtypes, vectorization, indexing, and clean tabular ops are critical for reliable data pipelines.

PYTHON

```
# code  
a
```

Deep Dive : np.delete — remove elements

- Removes elements at given indices
- Returns new array (original intact)
- axis for multidim; default flatten
- Indices in flat order
- np.delete vs masking (mask preferred often)
- Removes, doesn't replace
- Gap: index list any subset
- Keep-filter `a[mask]` faster many cases
- Data cleaning drop indices
- attributes remain

PYTHON

```
np.delete(a, [0,2,4])
```

PYTHON

```
np.delete(a, [0,2,4])    # remove positions 0,2,4 → new array
```

Set functions

- np.union1d
- np.intersect1d
- np.setdiff1d
- np.setxor1d
- np.in1d

Sets

Unordered collections of unique elements. Useful for membership tests and set operations (union/intersection).

Deep Dive : Set functions

- Set algebra on arrays
- union — all values both
- intersect — common values
- setdiff1d(a,b) — in a not in b (order matters)
- setxor — exactly one
- in1d → membership mask
- Returned sorted unique
- Data dedupe/merge analysis
- pandas merge similar
- vectorized set ops

PYTHON

```
m = np.array([1,2,3,4,5])
n = np.array([3,4,5,6,7])

np.union1d(m,n)
```

PYTHON

```
np.intersect1d(m,n)
```

np.clip

numpy.clip() function is used to Clip (limit) the values in an array.

<https://numpy.org/doc/stable/reference/generated/numpy.clip.html>

Why NumPy

Vectorized C-backed ops, contiguous memory, broadcasting. Essential for numerical work and ML arrays.

PYTHON

```
# code
a
```

Deep Dive : np.clip — limit values

- Clip values to range [min, max]
- Below min → min; above max → max
- amin/amax in order
- Outlier clipping (winsorization)
- Model outputs softmax no need

- input data scale bounds
- in-place variant: `np.clip(out=)`
- Gradient clipping concept
- data-safety transformation
- kwargs form notation common

PYTHON

```
np.clip(a, a_min=25, a_max=75)
```

PYTHON

```
np.clip(a, a_min=25, a_max=75) # 25 ≤ val ≤ 75
```

The closing eight — `swapaxes`, `uniform`, `count_nonzero`, `tile`, `repeat`, `allclose`

The final stubs round out the collection.

- `np.swapaxes(a, i, j)` exchanges two axes of an array; for 2D input it is the same as transposing, but it works on any pair of axes in higher dimensions.
- `np.random.uniform(low, high, size)` draws continuous random values evenly spread between `low` and `high` — useful when you need a custom range instead of `[0, 1)`.
- `np.count_nonzero(a)` counts how many elements are non-zero, or equivalently how many are True in a boolean mask — the fast way to tally matches.
- `np.tile(a, reps)` repeats the **whole array** to build a bigger block.
- `np.repeat(a, n)` repeats **each element** `n` times before moving on — the distinction from `tile` is worth memorising.
- `np.allclose(a, b)` returns True when two arrays are equal within a small tolerance; because floats rarely match exactly after computation, `allclose` is the community-standard comparison for test assertions.

Run each stub, predict the output, then check. Being able to choose between these six at the right moment is a strong sign the toolkit has sunk in.

Real-World and AI Engineering Relevance

Where this shows up in AI engineering

- `ndarray`, `dtype`, `shape` and `strides`
- array construction and manipulation
- vectorisation and broadcasting
- indexing, slicing and boolean masks
- reductions, performance and memory

EXAM AUR INTERVIEW PERSPECTIVE

Agar koi concept interview ya exam mein poochha jaye, to definition ke saath apna ek chhota practical example zaroor bolo. Sirf theory batane se confidence nahi dikhta - implementation dikhane se dikhta hai.

Quick Reference

Short, representative snippets collected from the unit's notebooks. Use them as syntax reminders, not as complete programs.

PYTHON

NP.ARRAY — BASIC

```
import numpy as np

a = np.array([1, 2, 3])           # 1D
b = np.array([[1, 2, 3], [4, 5, 6]]) # 2D
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]]) # 3D
```

PYTHON

DEEP DIVE : NUMPY VS PYTHON LISTS

```
# np.array
import numpy as np

a = np.array([1,2,3])
print(a)
```

PYTHON

DEEP DIVE : NUMPY VS PYTHON LISTS

```
import numpy as np
a = np.array([1,2,3]) # 1D
print(a)              # [1 2 3]
```

PYTHON

DEEP DIVE : MULTI-DIMENSIONAL ARRAYS

```
# 2D and 3D
b = np.array([[1,2,3],[4,5,6]])
print(b)
```

PYTHON

DEEP DIVE : MULTI-DIMENSIONAL ARRAYS

```
c = np.array([[[1,2],[3,4]],[[5,6],[7,8]]])
print(c)
```

PYTHON

DEEP DIVE : MULTI-DIMENSIONAL ARRAYS

```
b = np.array([[1,2,3],[4,5,6]]) # 2D (matrix)
c = np.array([[[1,2],[3,4]],[[5,6],[7,8]]]) # 3D (tensor)
```

PYTHON

DTYPE & ARANGE

```
np.array([1, 2, 3], dtype=float) # dtype explicitly set
np.arange(1, 11, 2) # 1,3,5,7,9
np.arange(16).reshape(2, 2, 2, 2)
```

PYTHON

DEEP DIVE : DTYPE & NP.ARANGE

```
# dtype
np.array([1,2,3],dtype=float)
```

Glossary

Numpy arrays vs Python sequences

- Same type elements (homogeneous) — fixed dtype

axis

0 = column-wise (vertical), 1 = row-wise (horizontal)

Ordering and selection

``np.sort`, `np.append`, `np.concatenate`, `np.unique`, `np.expand_dims`, `np.where`.`

Extremes and cumulative stats

``np.argmax`, `np.argmin`, `np.cumsum`, `np.cumprod`, `np.percentile`, `np.histogram`, `np.corrcoef`.`

Membership and shape surgery

``np.isin`, `np.flip`, `np.put`, `np.delete`.`

Set-style operations``np.union1d`, `np.intersect1d`, `np.setdiff1d`, `np.setxor1d`, `np.in1d`.`**Limits and extras**``np.clip`, plus `swapaxes`, `uniform`, `count_nonzero`, `tile`, `repeat`, `allclose`.`

Revision Checklist

Work through this list without looking at the notes. Anything you cannot explain aloud belongs back in the revision queue.

- ☐ Numpy Arrays Vs Python Sequences
- ☐ Creating Numpy Arrays
- ☐ `np.array` — basic
- ☐ Deep Dive : NumPy vs Python lists
- ☐ Deep Dive : Multi-dimensional arrays
- ☐ `dtype` & `arange`
- ☐ Deep Dive : `dtype` & `np.arange`
- ☐ `ones` / `zeros` / `random` / `linspace` / `identity`
- ☐ Why these builders matter
- ☐ Array attributes
- ☐ Deep Dive : Array attributes
- ☐ Changing Datatype
- ☐ Deep Dive : Changing datatype — `astype`
- ☐ Array operations — scalar & vector

☐ Deep Dive : Scalar operations (vectorized)☐ Array Functions☐ Array functions — axis☐ Deep Dive : Array functions — axis☐ Indexing & slicing☐ Deep Dive : Indexing & slicing☐ Iterating arrays☐ Deep Dive : Iterating arrays